NYCU-EE IC LAB – Spring 2026

Lab08 Exercise

Design: Coupling Network

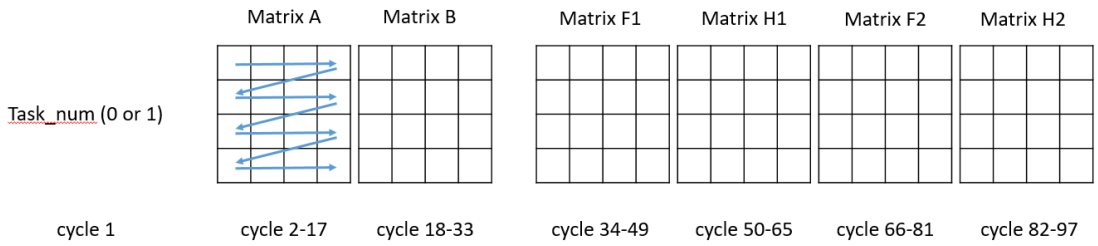
◆ Data Preparation

1. Extract test data from TA's directory:
   % tar xvf ~iclabTA01/Lab08_2026Spring.tar
2. The extracted LAB directory contains:
   a. EXERCISE/
   b. EXERCISE_wocg/
   c. JG/

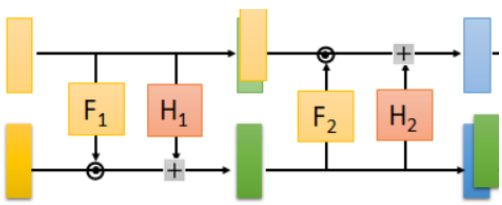
◆ Design Description

In this lab, you are required to implement two tasks. Task 0 is a forward coupling network, while Task 1 is a backward coupling network.

First, you will receive 97 consecutive cycles of input. The first cycle provides task_num (0 or 1). The following cycles, in order, provide matrix A, matrix B, matrix F1, matrix H1, matrix F2, and matrix H2. All inputs are given in raster-scan order. Note that all inputs are fed through in_data (8-bit). However, since task_num is only 1-bit, you only need to use in_data[0]. Matrix A and matrix B are 8-bit values, so use in_data[7:0]. Matrix F1, matrix H1, matrix F2, and matrix H2 are 4-bit values, so use in_data[3:0]. All operations in this lab are unsigned arithmetic.



Next, you need to perform the operations of the stacking coupling layer. The architecture is shown in the figure below.



Task_num = 0 (forward) :

First, you need to compute X1 and Y1.

- X1 is obtained by performing matrix multiplication between F1 and matrix A, then applying element-wise modulo 15, followed by adding 1 to each element.
- Y1 is obtained by performing matrix multiplication between H1 and matrix A, then applying element-wise modulo 256.

Next:

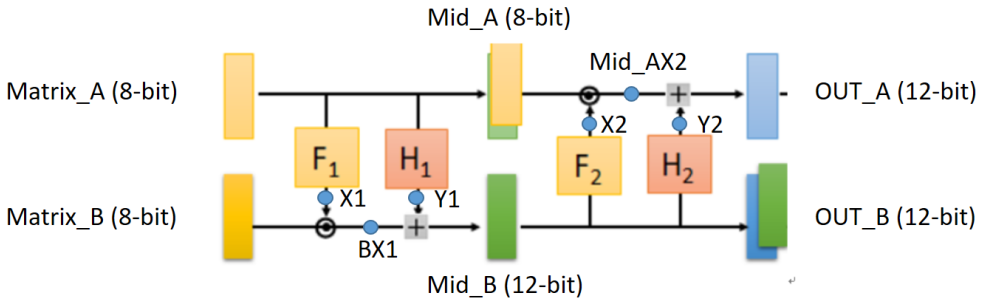
- BX1 is the element-wise (pointwise) multiplication (hadamard product) of X1 and matrix B.
- Mid_A is simply matrix A.
- Mid_B is obtained by performing a bitwise XOR between BX1 and Y1.

Then compute X2 and Y2:

- X2 is obtained by performing matrix multiplication between F2 and Mid_B, then applying element-wise modulo 15, followed by adding 1 to each element.
- Y2 is obtained by performing matrix multiplication between H2 and Mid_B, then applying element-wise modulo 256.

Finally:

- Mid_AX2 is the element-wise multiplication (hadamard product) of X2 and Mid_A.
- OUT_A is obtained by performing a bitwise XOR between Mid_AX2 and Y2.
- OUT_B is simply Mid_B.



X1 = (F1 x Matrix_A) % 15 + 1 (4-bit), Y1 = (H1 x Matrix_A) % 256 (8-bit)
BX1 = X1 ◉ Matrix_B (12-bit)
Mid_A = Matrix_A, Mid_B = BX1 ^ Y1

X2 = (F2 x Mid_B) % 15 + 1 (4-bit), Y2 = (H2 x Mid_B) % 256 (8-bit)
Mid_AX2 = X2 ◉ Mid_A (12-bit)
OUT_A = Mid_AX2 ^ Y2, OUT_B = Mid_B

Task_num = 1 (backward) :

You need to perform the inverse operation by tracing the computation backward.

First, compute X2 and Y2:

- X2 is obtained by performing matrix multiplication between F2 and matrix B, then applying element-wise modulo 15, followed by adding 1 to each element.
- Y2 is obtained by performing matrix multiplication between H2 and matrix B, then applying element-wise modulo 256.

Next:

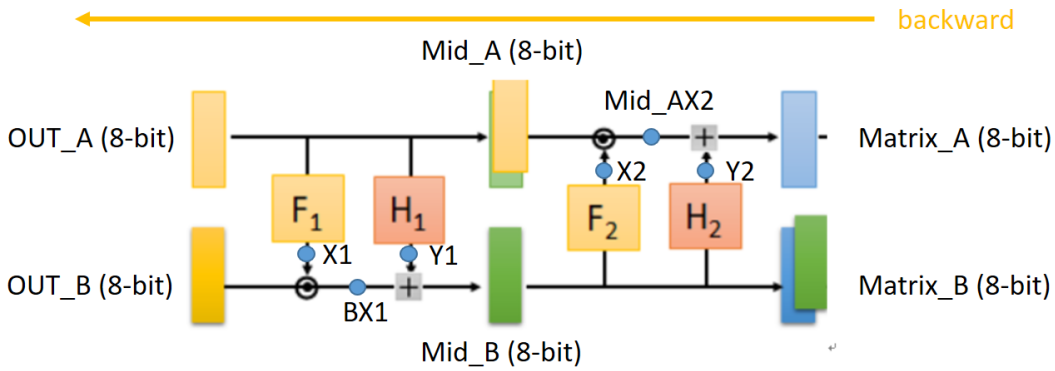
- Mid_AX2 is obtained by performing a bitwise XOR between matrix A and Y2.
- Mid_A is obtained by performing element-wise division of Mid_AX2 by X2.
- Mid_B is simply matrix B.

Then compute X1 and Y1:

- X1 is obtained by performing matrix multiplication between F1 and Mid_A, then applying element-wise modulo 15, followed by adding 1 to each element.
- Y1 is obtained by performing matrix multiplication between H1 and Mid_A, then applying element-wise modulo 256.

Finally:

- BX1 is obtained by performing a bitwise XOR between Y1 and Mid_B.
- OUT_A is simply Mid_A.
- OUT_B is obtained by performing element-wise division of BX1 by X1.

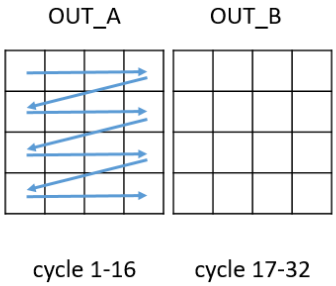


X2 = (F2 x Matrix_B) % 15 + 1 (4-bit), Y2 = (H2 x Matrix_B) % 256 (8-bit)
Mid_AX2 = Matrix_A ^ Y2 (8-bit)
Mid_A = Mid_AX2 / X2, Mid_B = Matrix_B

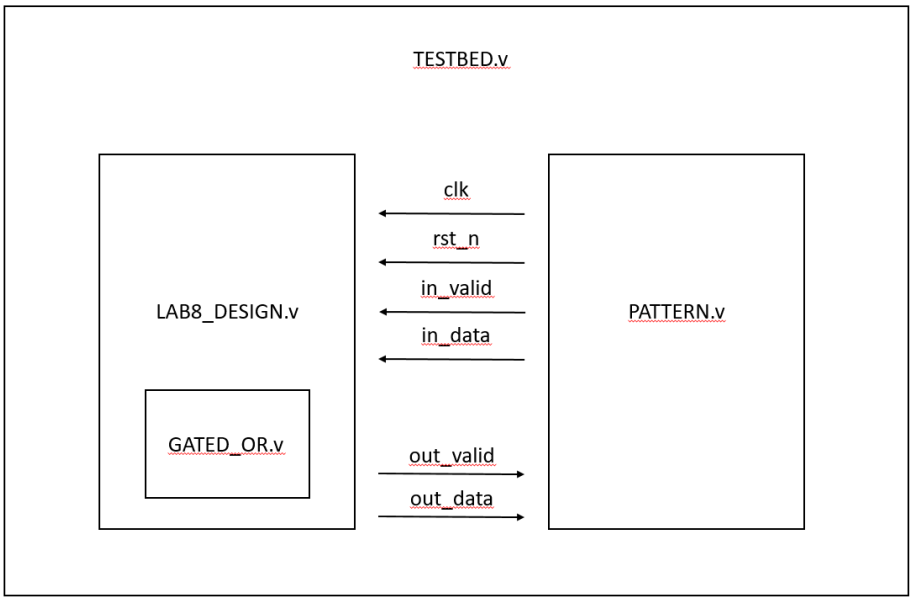
X1 = (F1 x Mid_A) % 15 + 1 (4-bit), Y1 = (H1 x Mid_A) % 256 (8-bit)
BX1 = Mid_B ^ Y1 (8-bit)
OUT_A = Mid_A, OUT_B = BX1 / X1

Output:

After computing OUT_A and OUT_B, you can start the output. Output the results in raster-scan order continuously for 32 cycles.



◆ System Architechure



◆ Lab Hint

In this lab, you need to design two version of module: LAB8_DESIGN_wocg.v & LAB8_DESIGN.v . RTL code must remain identical in both implementation and logic except for the clk_gating part; otherwise, the JG will fail.

◆ Inputs

I/O	Signal name	Bit Width	Description
Input	**clk**	1	Clock
Input	**rst_n**	1	Asynchronous active low reset
Input	**in_valid**	1	High when input signals are valid.
Input	**in_data**	8	**in_data** is valid when **in_valid** is high The in_data will be given in **97** cycles continuously.

◆ Outputs

I/O	Signal name	Bit Width	Description
output	**out_valid**	1	Should set to low after reset**.** Should set to high in **32** cycles continuously when your **out_data** is valid. Should set to low when your **out_data** is invalid.
output	**out_data**	12	Should set to low after reset**.** Output the resulting number series. The **out_data** should be given in **32** cycles continuously.

◆ Special Input

I/O	Signal name	Bit Width	Description
Input	**cg_en**	1	If cg_en is high, you should execute clock gating. Otherwise, if cg_en is low, you won't do clk gating.

◆ Specifications

1. Top module name：LAB8_DESIGN
2. Input pins ：**clk, rst_n, cg_en, in_valid, in_data[7:0]**
   Output pins：**out_valid, out_data[11:0]**
3. Use **asynchronous** reset active low architecture.
4. All your output register should be set zero after reset.
5. The input cg_en is in LAB8_DESIGN.v for enable signal of clock gating.
6. Changing clock period is prohibited **(fixed at 15ns)**.
7. The instance name of the clock gating cell (GATED_OR) should be GATED_XXX (e.g., GATED_out).

```
GATED_OR GATED_out (
    .CLOCK(clk), .SLEEP_CTRL(G_sleep_out),
    .RST_N(rst_n), .CLOCK_GATED(G_clock_out)
);
```

8. After synthesis, check the "LAB8_DESIGN.area" and "LAB8_DESIGN.timing" in the folder "Report". The area report is valid only when the slack in the end of "SP.timing" is **non-negative** and the result should be **MET**.
9. The next input will come in **2~5** cycles after your **out_valid** is pulled down.
10. The synthesis result **cannot** contain any **LATCH except for clocking gating latch.**
11. The synthesis result **cannot** contain any error**.**
12. The output loading is set to 0.05.
13. Input delay and output delay are 0.5*Clock Period.
14. You can't have timing violation in gate-level simulation
15. You design **can't use memory** in this lab.

16. You design **can't use DesignWare IP** in this lab.
17. The execution latency is limited in **200 cycles.**
18. The latency is the clock cycles between the falling edge of in_valid and the rising edge of out_valid in the same sequence of data.
19. **The total cell area should not be larger than 600,000 μm^2.**
20. The **out_valid** cannot overlap with **in_valid**.
21. The **out_valid** should set to high in **32** cycles continuously when your **out_data** is valid.
22. The **out_valid** should set to low when your **out_data** is invalid.
23. The **out_data** should be given in **32** cycles continuously.
24. **The redundant cycle is forbidden in this lab.**
25. Your design should have **at least 10% power reduction** from **cg_en-off to cg_en-on**, otherwise it will be treated as failed. $\frac{P_{cg_enoff} - P_{cg_enon}}{P_{cg_enoff}} \geq 10\%$
26. **Power report position: 04_PTPX/Report/**

Report Total Power Example:

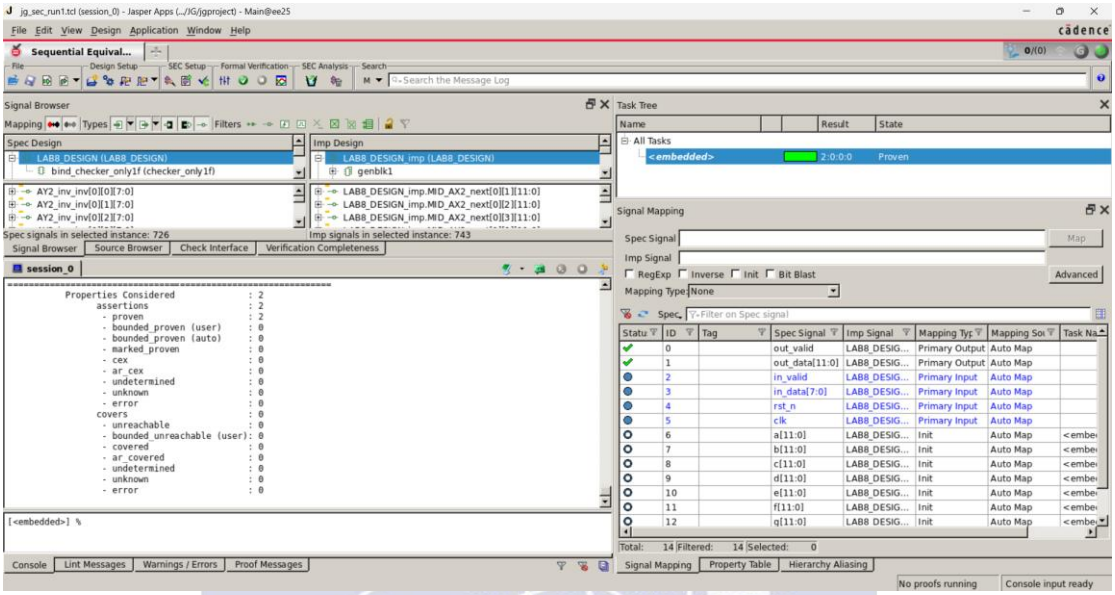
LAB8_DESIGN_POWER (w/o clock gating)	**LAB8_DESIGN _CG_POWER (with clock gating)**
Net Switching Power = 1.135e-03 (22.33%) Cell Internal Power = 3.933e-03 (77.40%) Cell Leakage Power = 1.348e-05 (0.27%) Intrinsic Leakage = 1.348e-05 Gate Leakage = 0.0000 --- Total Power = 5.082e-03 (100.00%) X Transition Power = 1.015e-05 Glitching Power = 0.0000 Peak Power = 0.0428 Peak Time = 16680	Net Switching Power = 1.013e-03 (28.28%) Cell Internal Power = 2.556e-03 (71.35%) Cell Leakage Power = 1.348e-05 (0.38%) Intrinsic Leakage = 1.348e-05 Gate Leakage = 0.0000 --- Total Power = 3.583e-03 (100.00%) X Transition Power = 1.015e-05 Glitching Power = 0.0000 Peak Power = 0.0373 Peak Time = 16680

27. The gate level simulation cannot include any timing violations without the *notimingcheck* command. You can ignore the warning about TFIPC and SDFCOM_CFTC.

```
Warning-[SDFCOM_CFTC] Cannot find timing check
LAB8_DESIGN_SYN.sdf, 203782
module: QDFFRBS, "instance: TESTBED.u_LAB8_DESIGN.X2_reg_1__3__3_"
  SDF Warning: Cannot find timing check $hold(posedge CK,posedge RB,...)
```

28. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
29. Don' t write Chinese comments or other language comments in the file you turned in.
30. **Any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.**

31. **For JG: demo will check whether out_valid and out_data are enabled (checked) in the right window, and whether both assertions and proven are equal to 2 in the left window. An error message showing "1 unmapped signal in imp" is normal—this corresponds to the cn_en signal and can be ignored.**



```
[<embedded>] % check sec -interface
primary_input       - FAILED - found 1 unmapped signal(s) in imp.
Overall interface check result - FAILED
[<embedded>] % assume LAB8_DESIGN_imp.cg_en==0
WARNING (WNL023): Expression sizes mismatch.
[<embedded>] % check sec -prove -bg
WARNING (WMTR018): cutpoint on a verification pair (primary_output, bbox_input, or mpram_port): "out_valid ->
LAB8_DESIGN_imp.out_valid".
WARNING (WMTR018): cutpoint on a verification pair (primary_output, bbox_input, or mpram_port): "out_data[11:0] -
> LAB8_DESIGN_imp.out_data[11:0]".
```

Console | Lint Messages | Warnings / Errors | Proof Messages

32. If you passed 01_RTL but encounter X (unknown) values during DFF data propagation caused by clk_gating in 03_gate, it is likely due to excessive fanout from the gating signal. It is recommended not to have multiple DFFs share the same clock gating.

Hint: use generate

33.demo time for JG ./01_run < 30 minutes, for JG ./02_run < 60 minutes.

◆ Grading Policy

- **Functionality (80%)：**
  - LAB8_DESIGN_wocg and LAB8_DESIGN：**70%**

    *(You can only get these points if you pass the gate level simulation **both**)*
  - JasperGold SEC check (5%)：**Run1 (5%) , Run2 (5%)**

(You can only get these points if you pass the gate level simulation. JG No 2nd demo chance.)

- **Performance (20%)：**

  Total Latency * (**Total Power** *(gated with CG)* ^ 2) * **Area**

◆ Note

1. **Please submit following files under 09_SUBMIT before 12:00 at noon on 05/04:**
   - **LAB8_DESIGN_wocg.v, LAB8_DESIGN.v**
   - If uploaded files violate the naming rule, you will get 5 deduct points.
   - In this lab, you can't adjust your clock cycle time.
   - The 2nd demo deadline is **12:00 at noon on 05/06** .
   - Check whether there is any wire / reg / submodule name called "error", "fail", "pass", "congratulation", if you used, you will fail the lab.
2. Template folders and reference commands for EXERCISE folder:

   ■ 01_RTL/ (for RTL simulation):
   ./01_run_vcs_rtl (cg_en = 0)
   ./01_run_vcs_rtl_CG (cg_en = 1)

   ■ 02_SYN/ (for Synthesis):
   ./clk_dc_shell (for CG) You need to do this first before ./01_run_dc_shell
   ./01_run_dc_shell
   (Check the design which contains Latch and Error or not in syn.log)

   ■ 03_GATE_SIM/ (Gate Level simulation):
   ./01_run_vcs_gate (cg_en = 0)
   ./01_run_vcs_gate_CG (cg_en = 1)

   ■ 04_PTPX/ (PrimeTime power analysis):
   ./01_run_ptpx
   ./01_run_cg_ptpx

   ■ JG/ (JasperGold SEC)
   ./01_run
   ./02_run

   ■ You can key in ./09_clean_up to clear all log files and dump files in each

folder

■ 09_SUBMIT/ (submit your files):

./00_tar

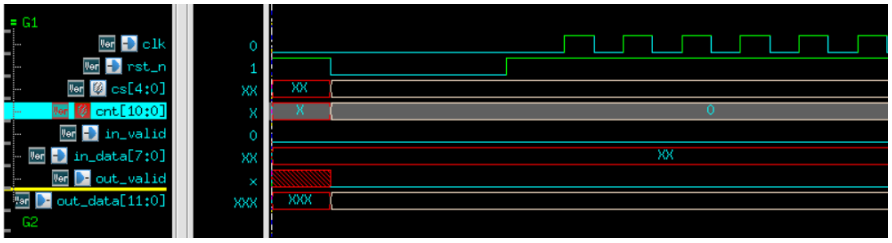
./01_submit 1st_demo/2nd_demo

./02_check 1st_demo/2nd_demo.

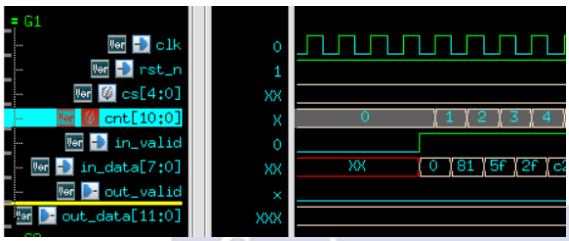


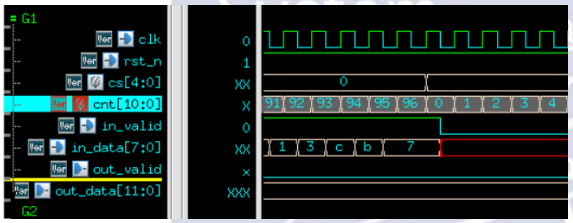
Waveform Example

1. Reset Signal



2. Input





3. Output

